OrganoidAgent: An Agent-Orchestrated Platform for Reproducible Organoid Dataset Analysis and Computational Phenotyping

Rongzhou Chen Zhenyu Wu SHaohua Ma

March 2, 2026

Abstract

Organoid experiments increasingly generate large, heterogeneous collections of brightfield images, tabular readouts, and single-cell analysis files, yet many groups still rely on fragmented scripts or manual inspection for quality control and early phenotyping. We present OrganoidAgent, an agent-orchestrated software system for organizing, previewing, and analyzing multimodal organoid datasets in a reproducible workflow. OrganoidAgent combines a lightweight Tornado backend with a progressive web interface and task-specific analysis modules for tabular data preview, archive introspection, TIFF rendering, and single-cell `.h5ad` summarization with automated two-dimensional embedding visualization. The system exposes standardized APIs for dataset listing, category-based filtering, preview generation, and archive extraction, enabling rapid triage of experimental outcomes before deeper modeling. We describe the architecture, implementation, and an evaluation protocol designed for translational organoid workflows, including quality control gates and pathway-specific analysis entry points. This manuscript provides a submission-ready technical foundation for a Nature-family methods paper and includes explicit placeholders for experiment-specific quantitative benchmarking, biological validation, and ablation analyses.

Main

1 Introduction

Organoid models are increasingly used to study tissue development, disease mechanisms, and therapeutic response under physiologically relevant conditions. In practice, organoid studies produce multimodal artifacts at different scales: microscopy image stacks, compressed archives, tabular summaries, and single-cell feature matrices. Despite growing data volume, routine analysis pipelines remain labor-intensive and often brittle, with limited standardization across experiments.

Agent-oriented systems can improve this bottleneck when they are used as orchestration layers over explicit, auditable computational tools rather than opaque black-box decision makers. We therefore designed OrganoidAgent as a practical platform to streamline data intake, dataset browsing, preview generation, and first-pass computational phenotyping for organoid experiments.

The core goals were: (i) reduce time from raw data acquisition to interpretable previews, (ii) provide a uniform API surface across heterogeneous file types, (iii) support reproducible quality-control and analysis handoff, and (iv) maintain low operational complexity for laboratory deployment.

2 System Overview

OrganoidAgent is implemented as a Python backend (Tornado) with a progressive web application frontend. The backend exposes HTTP endpoints for dataset-level and file-level operations, while the frontend enables rapid interactive exploration. The system is organized around five core capabilities:

1. **Dataset inventory and file indexing.** Recursive scanning of dataset folders with file count and storage summaries.
2. **Type-aware preview generation.** Automatic handling of tables (CSV/TSV/Excel), analysis files (H5AD/HDF5), images (including TIFF), archives (ZIP/TAR/TGZ), and compressed text (GZ).
3. **Single-cell analysis preview.** Fast summary extraction from `.h5ad` files (observations, variables, metadata keys) with embedding-based scatter previews (UMAP/t-SNE/PCA when available; sampled PCA fallback otherwise).
4. **Archive introspection and extraction.** Entry listing with optional first-image preview and API-driven extraction into tracked subfolders.
5. **Category routing for analysis stages.** API filters to group files into dataset, segmentation, feature, and analysis categories for downstream workflows.

3 Implementation Details

3.1 Backend architecture

The backend is centered on deterministic, stateless API handlers that wrap utility functions for file classification, safe path resolution, and preview rendering. A strict path sanitizer ensures requests remain inside the configured dataset root. Preview artifacts are cached using hash-derived filenames in a dedicated cache directory, minimizing repeated compute and improving responsiveness for interactive use.

3.2 Analysis-specific preview methods

For table files, OrganoidAgent returns bounded row previews and column metadata. For TIFF microscopy files, a representative frame is selected and normalized for visualization, with size-constrained thumbnails generated for browser display. For single-cell `.h5ad` files, the backend attempts to reuse available embeddings from `obsm`; if absent, it computes a sampled PCA preview to provide a fast structural summary. These previews are intended for triage and exploratory QA, not for final inferential reporting.

3.3 Frontend interaction model

The web interface consumes the backend APIs to support dataset browsing, file inspection, and type-specific previews from a unified surface. This design allows wet-lab and computational users to align on the same intermediate artifacts before formal statistical analysis.

4 Evaluation Framework for Biological Studies

To support publication-grade validation, we define an evaluation framework that separates engineering reliability from biological performance:

1. **Operational reliability.** API success rate, median preview latency by file type, cache hit ratio, and extraction completion rate.
2. **Data quality and triage utility.** Fraction of files receiving interpretable previews, error taxonomy, and manual agreement on preview correctness.

3. **Biological analysis readiness.** Agreement between rapid previews and full offline pipelines on key endpoints (for example, dimensional structure in single-cell datasets).

4. **Workflow impact.** Reduction in analyst time-to-first-insight relative to ad hoc script-based workflows.

[TODO: Insert quantitative benchmarking results on internal datasets, with confidence intervals and replicate definitions.]

5 Representative Use Cases

5.1 Multimodal dataset triage

A common entry point is a mixed dataset containing microscopy files, archive bundles, and analysis matrices. OrganoidAgent enables rapid identification of actionable files by category, immediate visual checks on image quality, and lightweight statistical context for single-cell objects.

5.2 Segmentation and feature-engineering handoff

Category-specific API routes separate candidate segmentation inputs (image/archive) from feature tables and analysis matrices, supporting modular downstream pipelines while maintaining traceability to original data assets.

5.3 Archive-first ingestion

When data arrive as compressed bundles, OrganoidAgent can inspect archive contents, preview representative image entries, and extract files into deterministic locations to simplify reproducible preprocessing.

6 Discussion

OrganoidAgent addresses a practical and under-served stage of organoid analysis: rapid, reproducible conversion of heterogeneous raw files into inspectable computational artifacts. The current implementation prioritizes robust file handling, preview generation, and workflow orchestration.

This foundation is compatible with future integration of segmentation, cell tracking, phenotyping, and predictive modeling modules.

The framework also clarifies the role of agentic systems in biomedical data science. Rather than replacing domain expertise, the agent layer standardizes routine operations, surfaces quality signals early, and creates a consistent interface between experimental and computational teams.

Key limitations include dependence on local file organization conventions, variable metadata quality across studies, and the need for study-specific validation before inferential claims. For Nature-family publication standards, future versions should include controlled ablations, cross-batch generalization tests, and prospective biological endpoint validation.

7 Methods

7.1 Software environment

OrganoidAgent is implemented in Python with Tornado for web serving. Optional dependencies include pandas (table preview), anndata plus numpy (single-cell preview), Pillow (image rendering), and tifffile (TIFF handling). The application can be run locally on standard laboratory workstations.

7.2 File typing and routing

File classification uses extension-based routing with explicit support for image, table, analysis, archive, compressed-text, and document types. Unsupported files remain accessible via direct download.

7.3 Preview generation

Table previews are row-limited for responsiveness. TIFF previews use frame normalization to map intensity ranges for browser display. Single-cell previews extract metadata dimensions and attempt embedding reuse before fallback PCA with random subsampling.

7.4 Security and data locality

The backend uses sanitized relative paths and resolves all requests under the dataset root to prevent path traversal. All processing is local to the deployment environment.

8 Data Availability

The data used to generate publication figures and benchmark tables should be deposited in a public repository at the time of submission. **[TODO: Insert accession IDs and persistent links.]**

9 Code Availability

OrganoidAgent source code is available in the project repository. **[TODO: Insert public repository URL and tagged release hash used for the manuscript.]**

10 Acknowledgements

We thank collaborators and laboratory members for discussions on organoid workflow requirements and evaluation priorities. **[TODO: Insert funding sources and grant numbers.]**

11 Author Contributions

R.C., Z.W., and S.M. conceptualized the study. R.C. and Z.W. implemented the software and performed computational analysis. S.M. supervised the project. All authors contributed to manuscript writing and revision.

12 Competing Interests

The authors declare no competing interests.

References

[1] Huch, M. & Koo, B.-K. Modeling mouse and human development using organoid cultures. *Development* **142**, 3113–3125 (2015).

[2] Fatehullah, A., Tan, S. H. & Barker, N. Organoids as an in vitro model of human development and disease. *Nat. Cell Biol.* **18**, 246–254 (2016).

[3] Wolf, F. A., Angerer, P. & Theis, F. J. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biol.* **19**, 15 (2018).

[4] FriendFeed. Tornado Web Server and Tools. https://www.tornadoweb.org/ (accessed March 2, 2026).

[5] Virshup, I. *et al.* The anndata ecosystem for annotated data matrices. https://anndata.readthedocs.io/ (accessed March 2, 2026).

[6] Stringer, C. *et al.* Cellpose: a generalist algorithm for cellular segmentation. *Nat. Methods* **18**, 100–106 (2021).